科目B演習　5問ミニテスト 第6回　問題と解説

教員予習用（学生配布不可）／標準試験時間 25分／全5問

問	分野	テーマ	難易度	正解
問1	アルゴリズム	二つの整列済み配列の併合（マージ）	基礎	**イ**
問2	アルゴリズム	回文の判定	基礎	**ウ**
問3	アルゴリズム	累積和による区間の和	基礎〜応用	**エ**
問4	アルゴリズム	2分木の高さ（再帰）	応用	**ア**
問5	情報セキュリティ	委託先・クラウド利用時の責任分界	基礎〜応用	**ウ**

授業メモ

問1　問題　　アルゴリズム／二つの整列済み配列の併合（マージ）／基礎

問1　次の記述中の［　a　］と［　b　］に入れる正しい答えの組合せを，解答群の中から選べ。ここで，配列の要素番号は1から始まる。

関数 merge は，それぞれ昇順に整列された整数型の配列 x と y を受け取り，x と y の全ての要素を昇順に並べた配列を返す。x の要素と y の要素の値が等しいときは，x の要素を先に追加する。

〔プログラム〕

```
○整数型の配列: merge(整数型の配列: x, 整数型の配列: y)
  整数型の配列: c ← {}    // 要素数0の配列
  整数型: i ← 1
  整数型: j ← 1
  while ((i ≦ xの要素数) and (j ≦ yの要素数))
    if ( a )
      cの末尾 に x[i] を追加する
      i ← i + 1
    else
      cの末尾 に y[j] を追加する
      j ← j + 1
    endif
  endwhile
  while (i ≦ xの要素数)
    cの末尾 に x[i] を追加する
    i ← i + 1
  endwhile
  while (j ≦ yの要素数)
    cの末尾 に y[j] を追加する
    j ← j + 1
  endwhile
  return c
```

関数 merge を merge({2, 5, 8}, {1, 5, 6, 9}) として呼び出すと，最初の while の繰返しは［　b　］回実行される。

解答群

	a	b
ア	x[i] ＜ y[j]	6
イ	x[i] ≦ y[j]	6
ウ	x[i] ≦ y[j]	7
エ	x[i] ≧ y[j]	3

問1　解説　　正解:イ

正解:**イ**

■ 出題のねらい

マージソートの基本となる「併合」の処理で，二つの配列の先頭どうしを比較して小さい方から取り出す流れと，値が等しいときの扱いを理解しているかを確認します。

■ 空欄a

x[i] と y[j] のうち小さい方を c に追加します。値が等しいときは x の要素を先に追加するので，等しいときも x 側に進む **x[i] ≦ y[j]** が正解です。

■ 空欄b（トレース）

最初の while のトレース

回	x[i]	y[j]	追加する値	c
1	2	1	y の 1	{1}
2	2	5	x の 2	{1, 2}
3	5	5	x の 5	{1, 2, 5}
4	8	5	y の 5	{1, 2, 5, 5}
5	8	6	y の 6	{1, 2, 5, 5, 6}
6	8	9	x の 8	{1, 2, 5, 5, 6, 8}

6回目の後で i ＝ 4 となり x の要素を使い切るので，最初の while を抜けます。繰返しは **6回**です。残った y の 9 は3番目の while で追加されます。

■ ポイント

最初の while はどちらかの配列を使い切った時点で終わります。全要素の個数（7）だけ繰り返すわけではありません。残りは後の while でまとめて追加します。

■ 選択肢ごとの解説

ア	結果の並びは昇順になりますが，3回目で値が等しい（5 と 5）ときに y の要素を先に追加してしまい，「x の要素を先に」という条件を満たしません。
イ（正解）	等しいときも x を先に追加し，繰返しは6回です。
ウ	要素の総数7回と考えた誤りです。y の 9 は最初の while の後で追加されます。
エ	大きい方（x 側）を先に追加してしまい，{2, 5, 8, 1, 5, 6, 9} という昇順でない配列になります。

問2　問題　　アルゴリズム／回文の判定／基礎

問2　次のプログラム中の［　a　］と［　b　］に入れる正しい答えの組合せを，解答群の中から選べ。ここで，配列の要素番号は1から始まる。

関数 isPalindrome は，1文字以上の文字を格納した文字型の配列 s を受け取り，s が先頭から読んでも末尾から読んでも同じ並び（回文）であれば true を，そうでなければ false を返す。

例えば，s が {"l", "e", "v", "e", "l"} や {"a", "b", "b", "a"} のときは true を，{"a", "b", "c", "a"} のときは false を返す。

〔プログラム〕

```
○論理型: isPalindrome(文字型の配列: s)
  整数型: left ← 1
  整数型: right ← sの要素数
  while (［ a ］)
    if (s[left] が s[right] と等しくない)
      return false
    endif
    left ← left + 1
    right ← ［ b ］
  endwhile
  return true
```

解答群

	a	b
ア	left ＜ right	right + 1
イ	left ≠ right	right − 1
ウ	left ＜ right	right − 1
エ	right ＜ left	right − 1

問2　解説　　正解：ウ

正解：**ウ**

■ 出題のねらい

配列の両端から中央に向かって二つの添字を動かす処理で，添字の更新方向と，要素数が奇数・偶数の両方で正しく終了する条件を判断できるかを確認します。

■ 解き方

left は先頭から右へ，right は末尾から左へ動かして，対応する文字を比べます。したがって b は **right − 1** です。

二つの添字が出会うか，すれ違ったら比較は全て終わっています。繰返しを続ける条件は **left ＜ right** です。

正しいプログラムの動き

s	要素数	比べる組	終了時の left, right
level	5（奇数）	(1, 5), (2, 4)	3, 3（出会う）
abba	4（偶数）	(1, 4), (2, 3)	3, 2（すれ違う）

■ ポイント

添字を両端から動かす処理では，**要素数が奇数のときと偶数のときの両方**を必ず確かめましょう。片方だけでは条件の誤りに気付けないことがあります。

■ 選択肢ごとの解説

ア	right が増えてしまい，2回目の比較で s[6] のように範囲外を参照します。
イ	要素数が奇数なら left ＝ right で止まりますが，偶数（abba）では left と right が等しくならずにすれ違い，範囲外を参照します。
ウ（正解）	両端から中央に向かって比べ，出会うかすれ違った時点で終了します。
エ	最初から条件が偽なので一度も比較せず，常に true を返します。

問3　問題　　アルゴリズム／累積和による区間の和／基礎〜応用

問3　次のプログラム中の［　a　］と［　b　］に入れる正しい答えの組合せを，解答群の中から選べ。ここで，配列の要素番号は1から始まる。

整数型の配列 data について，「l 番目から r 番目までの要素の和」を何度も求めたい。そこで，前もって手続 makePrefix で配列 p を作っておき，関数 rangeSum で和を求める。

p の要素数は dataの要素数 ＋ 1 であり，p[k] には data の先頭から k − 1 番目までの要素の和が格納される（p[1] は 0）。例えば，data が {3, 1, 4, 1, 5} のとき，p は {0, 3, 4, 8, 9, 14} となり，rangeSum(2, 4) は 1 + 4 + 1 ＝ 6 を返す。ここで，1 ≦ l ≦ r ≦ dataの要素数 とする。

〔プログラム〕

```
大域：整数型の配列：p

○makePrefix(整数型の配列：data)
  整数型：i
  p ← {(dataの要素数 + 1)個の 0}
  for（i を 1 から dataの要素数 まで 1 ずつ増やす）
    p[i + 1] ← [ a ]
  endfor

○整数型：rangeSum(整数型：l，整数型：r)
  return [ b ]
```

解答群

	a	b
ア	p[i] + data[i + 1]	p[r + 1] − p[l]
イ	p[i] + data[i]	p[r] − p[l]
ウ	p[i] + data[i]	p[r + 1] − p[l − 1]
エ	p[i] + data[i]	p[r + 1] − p[l]

問3　解説　　正解：エ

正解：**エ**

■ 出題のねらい

累積和（先頭からの和）を前もって作っておくと，区間の和を引き算1回で求められることと，添字が1ずれる関係を正しく扱えるかを確認します。

■ 空欄a

p[i + 1] は「data の 1 番目から i 番目までの和」です。これは p[i]（1 番目から i − 1 番目までの和）に data[i] を加えたものなので，**p[i] + data[i]** です。

data = {3, 1, 4, 1, 5} のときの p

k	1	2	3	4	5	6
data[k]	3	1	4	1	5	—
p[k]	0	3	4	8	9	14

■ 空欄b

l 番目から r 番目までの和は，「1 番目から r 番目までの和」p[r + 1] から「1 番目から l − 1 番目までの和」p[l] を引いたものです。よって **p[r + 1] − p[l]** です。rangeSum(2, 4) は p[5] − p[2] = 9 − 3 = 6 となり，例と一致します。

■ ポイント

毎回 l から r まで足すと区間の長さに比例した時間がかかりますが，累積和を使えば1回の引き算で求められます。p[1] を 0 にしておくのは，l = 1 のときも同じ式で計算できるようにするためです。

■ 選択肢ごとの解説

ア	1番目の要素が足されず，i = 5 のときは data[6] という範囲外を参照します。
イ	p[r] は r − 1 番目までの和なので，r 番目の要素が含まれません。rangeSum(2, 4) は 5 になります。
ウ	l = 1 のとき p[0] を参照してしまい範囲外です。また l ≧ 2 でも l − 1 番目の要素が余分に含まれます。
エ（正解）	累積和の作り方も区間の和の求め方も正しい組合せです。

問4　問題　　アルゴリズム／2分木の高さ（再帰）／応用

問4　次の記述中の［　a　］と［　b　］に入れる正しい答えの組合せを，解答群の中から選べ。

関数 height は，2分木の節 n を受け取り，n を根とする部分木の高さを返す。ここで，木の高さは根から葉までの経路のうち最も長いものに含まれる節の個数とし，空の木（n が未定義）の高さは 0 とする。

クラス Node は2分木の節を表す。関数 max(x, y) は x と y のうち大きい方の値を返す。

表1　クラス Node のメンバ変数

メンバ変数	型	説明
left	Node	左の子の参照。左の子がないときは未定義
right	Node	右の子の参照。右の子がないときは未定義

〔プログラム〕

```
○整数型: height(Node: n)
  if（n が 未定義）
    return 0
  endif
  return ［ a ］
```

図1の2分木の根 A を引数として height を呼び出すと，戻り値は 4 となる。このとき，関数 height は，最初の呼出しも含めて［　b　］回呼び出される。

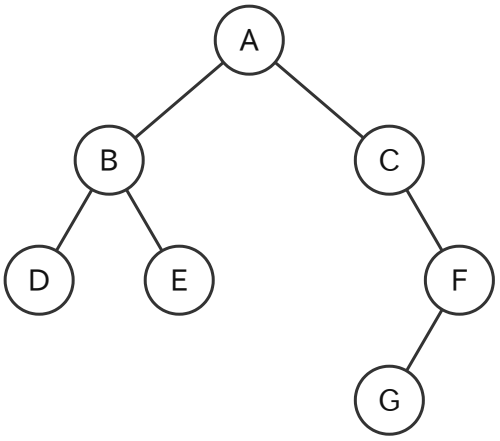


図1　2分木の例

解答群

	a	b
ア	max(height(n.left), height(n.right)) + 1	15
イ	max(height(n.left), height(n.right)) + 1	7
ウ	height(n.left) + height(n.right) + 1	15
エ	height(n.left) + height(n.right) + 1	7

問4　解説　　正解:ア

正解:**ア**

■ 出題のねらい

再帰で木の性質を求めるときに「子の結果をどう組み合わせるか」と，未定義（空の子）に対しても呼出しが行われることを理解しているかを確認します。

■ 空欄a

節 n を根とする部分木の高さは，左右の部分木のうち高い方の高さに，n 自身の 1 を加えたものです。よって **max(height(n.left), height(n.right)) + 1** です。

各節の高さ

節	左の高さ	右の高さ	高さ
D, E, G	0	0	1
F	1（G）	0	2
B	1（D）	1（E）	2
C	0	2（F）	3
A	2（B）	3（C）	4

■ 空欄b

height は節が未定義でも呼び出され，そこで 0 を返して終わります。節が7個あり，それぞれの節から左右2回ずつ呼び出すので，未定義の子に対する呼出しも含めて数えます。

節に対する呼出し 7 回 ＋ 未定義の子に対する呼出し 8 回（D の左右，E の左右，C の左，F の右，G の左右）＝ **15回** です。一般に，節が N 個の2分木では 2N + 1 回になります。

■ ポイント

再帰関数の呼出し回数を数えるときは，「終了条件で即座に戻る呼出し」も1回と数えることを忘れないようにしましょう。

■ 選択肢ごとの解説

ア（正解）	高い方の部分木の高さに1を加え，未定義の子への呼出しも含めて15回です。
イ	式は正しいですが，未定義の子に対する呼出し（8回）を数えていません。
ウ	左右の高さを足すと節の個数を数える関数になり，A に対しては 7 を返します。
エ	式が節の個数を求めるものになっており，呼出し回数も未定義の子の分を数えていません。

問5　問題　　情報セキュリティ／委託先・クラウド利用時の責任分界／基礎～応用

問5　C社は従業員120名の雑貨の通信販売会社であり，自社のECサイト（以下，Cサイトという）で商品を販売している。Cサイトの構成と運用の分担は図1のとおりである。

・Cサイトは，E社が提供するIaaS上の仮想サーバで稼働している。E社は，データセンタの設備，物理サーバ，ネットワーク機器及び仮想化基盤（ハイパーバイザ）を管理する。仮想サーバのOS及びその上で動作するソフトウェアは，利用者が管理する。
・仮想サーバのOS，Webサーバソフトウェア及びDBMSの導入と更新は，C社の情報システム課が行う。
・会員登録機能や商品検索機能などのWebアプリケーションのプログラムの開発と保守は，F社に委託している。F社との契約には，Webアプリケーションのプログラムの脆弱性の修正が含まれる。

図1　Cサイトの構成と運用の分担

C社は，セキュリティ診断会社のG社にCサイトの脆弱性診断を依頼した。G社からの指摘事項は表1のとおりであった。

表1　G社からの指摘事項

項番	指摘事項
(一)	会員登録画面の入力値をそのままSQL文に組み込んでおり，SQLインジェクションが可能である。
(二)	Webサーバソフトウェアのバージョンが古く，既に修正版が公開されている既知の脆弱性が残っている。
(三)	Cサイトが稼働している仮想化基盤のソフトウェアについて，提供元から脆弱性情報と修正版が公表されているが，まだ適用されていない。

設問　表1中の各指摘事項について，脆弱性に対処すべき組織の組合せとして，適切なものを解答群の中から選べ。

解答群

	(一)	(二)	(三)
ア	F社	F社	E社
イ	F社	C社	C社
ウ	F社	C社	E社
エ	C社	C社	E社
オ	F社	E社	E社
カ	C社	F社	E社

問5　解説　　正解：ウ

正解：**ウ**

■ 出題のねらい

業務委託やクラウドサービスを利用するときに，システムのどの層をどの組織が管理するか（責任分界）を読み取り，指摘された脆弱性ごとに対処すべき組織を判断できるかを確認します。科目Bの公開問題でも，委託先やクラウド事業者との役割分担を問う問題が出題されています。

■ 解き方

まず，指摘された脆弱性がシステムの**どの層**にあるかを判断し，次に図1からその層を**誰が管理しているか**を読み取ります。

Cサイトの責任分界

層	管理する組織	根拠（図1）
Webアプリケーションのプログラム	F社	開発と保守を委託し，脆弱性の修正も契約に含まれる
OS，Webサーバソフトウェア，DBMS	C社（情報システム課）	導入と更新を情報システム課が行う
仮想化基盤，物理サーバ，ネットワーク機器	E社	IaaSの提供者が管理する

■ 各項番の判定

項番	脆弱性のある層	対処する組織
(一)	Webアプリケーションのプログラム	F社
(二)	Webサーバソフトウェア（ミドルウェア）	C社
(三)	仮想化基盤	E社

■ ポイント

「Web」という言葉に引きずられて (二) を委託先のF社と考えないように注意します。Webサーバソフトウェアは，F社が作るプログラムではなく，C社が導入・更新しているソフトウェアです。また，IaaSでは基盤はクラウド事業者が管理しますが，**OSから上の層は利用者の責任**になります。SaaS，PaaS，IaaSで利用者が管理する範囲が異なることも合わせて確認しておきましょう。

■ 選択肢ごとの解説

ア	(二) のWebサーバソフトウェアはF社が開発するプログラムではなく，C社の情報システム課が導入と更新を行っています。
イ	(三) の仮想化基盤はIaaSの提供者であるE社が管理しており，C社は修正版を適用できません。
ウ（正解）	各脆弱性のある層と，図1の分担が正しく対応しています。
エ	(一) のプログラムの脆弱性の修正はF社との契約に含まれています。
オ	IaaSでは，OSから上の層（Webサーバソフトウェアを含む）は利用者であるC社が管理します。
カ	(一) と (二) の担当が入れ替わっています。